CS3230 : Design and Analysis of Algorithms (Spring 2014)

Tutorial Set #11 (the last tutorial)

[For discussion during Week 13]

OUT: 11-Apr-2014 **Tutorials:** Mon & Thu, 14 & 17 Apr 2014

Prepare your answers to all the D-Problems in every tutorial set.

When preparing to present your answers,

- Think of a CLEAR EXPLANATION
- Illustrate with a good worked example;
- Describe the main ideas,
- Can you sketch why the solution works;
- Give analysis of running time, if appropriate
- Can you think of other (perhaps simpler) solutions?

In CS3230, you learn to develop high-level abstractions when describing algorithms. Try not to speak in ML/AL (machine/assembly language) or "for (j=0; j<n; j++) do". Instead give names to your sets (of objects or things or data structures), talk about depth-first search, binary search, traverse the graph, sort the set, use a priority queue, etc. You are no longer in CS1010, CS1020, CS2010 or CS2020. Speak with greater sophistication, and at a higher level of abstraction.

Routine Practice Problems *-- do not turn these in -- but make sure you know how to do them.*

R1. (Exercise 35.1-1 of CLRS, slightly edited) Worst case instance for APPROX-MVC
Give an example for a graph for which APPROX-MVC (which is a 2-approximation algorithm) always gives solution that is exactly 2 times the cost of the optimal solution.

Answer:
R1. Just give a graph with E edges where no two edges in this graph share a common endpoint. The optimal MIN-VERTEX-COVER should be E/2 whereas the solution reported by APPROX-MVC is always E, i.e. exactly 2 times the cost of the optimal solution. To answer such question, one has to really understand what APPROX-MVC is doing.

Discussion-Problems -- Solve these problems and present them in tutorial class.

D1. (Lecture 8 review + Lecture 12 review, Exercise 35.2-2 of CLRS, slightly edited)
Show that TSP-Metric is NP-Complete

a). Show how in polynomial time we can transform one instance of the standard TRAVELING-SALESMAN-PROBLEM (TSP) into another instance whose cost function satisfies the triangle inequality (TSP-METRIC), i.e. TSP $\leq_p$ TSP-METRIC. The two instances must have the same set of optimal tour and the transformation must be done in polynomial time.

Hint: Consider adjusting the weight but do not change anything else (this question is quite hard if this hint is not mentioned).

Solution:

Let G(V, E) be the TSP graph and the weight of edge (u, v) in E is denoted by c(u, v).
Let W be the sum of all edge weights in E.
Let G'(V, E') be the TSP-METRIC graph which has the same set of vertices V and E' = E, but for each edge (u, v) in E', the weight is c(u, v) + W. This transformation is polynomial as we just need to explore O(E) edges to do so.

This transformation causes the weight of edges in G' satisfy triangle inequality as c(u, w)+W ≤ c(u, v)+W + c(v, w)+W (clear, we have two Ws in the RHS vs one W in the LHS). This transformation does <u>not</u> change the optimal tour, just that the cost increases by additional value of |V|*W.

b). Explain why such a polynomial-time transformation does not contradict the statement that TSP does not have finite approximation ratio, assuming that P ≠ NP.

Solution:

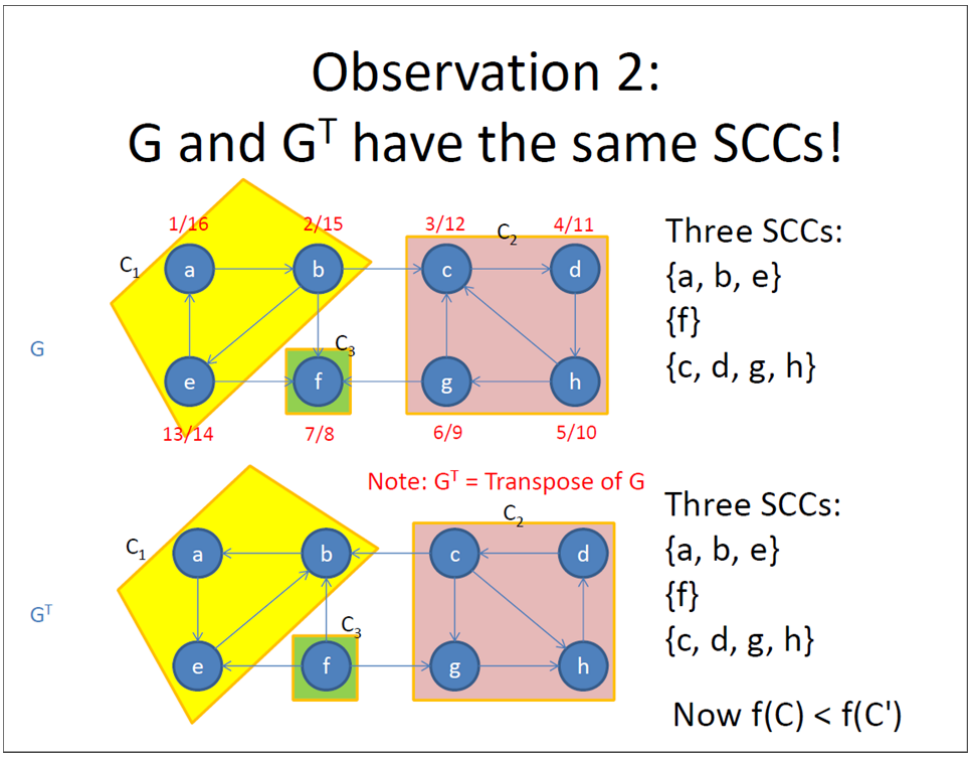
The 2-approximation algorithm for G' that satisfies triangle inequality does not translate to a constant approximation ratio for G.

One example: Just draw a small instance of V = 3 vertices with edge weight {1,2,X} where X is > 1+2. Since X is a variable, we can vary it. Let's just pick X = 1M-3 (1M is 1 million). Therefore W = 1M. We have edge 1M+1, 1M+2, and 2M-3. Now these 3 edge weights satisfy triangle inequality. The optimal tour cost in G' is at most 2(1M+1+1M+2+2M-3) = 2*4M = 8M. When we subtract 3*1M from this value, we still have a large value of 8M-3M = 5M, which is 5 times larger than 1+2+1M-3 = 1M, not at most 2 times larger.*

D2. (Lecture9 review, another Kosaraju's algorithm variant?)

Upon seeing this slide 55 of Lecture note 09 (copied below), one student J from CS3230 claims that he can simplify Kosaraju's algorithm as follows:

1. Call DFS(G) to find the finishing time f(u) for all vertices u in G
2. Run DFS(G) again on the same G (i.e. no need to create G^T)
   but in **increasing finishing time f(u).**



Student J claims that his variant of Kosaraju's algorithm is correct and simpler.
Using the same example from slide 55 above, student J claims that the his Kosaraju's algorithm variant will first explore vertex f in G (with finishing time 8) to obtain SCC {f}, then explore vertex g in G (with finishing time 9) to obtain SCC {g, c, d, h}, finally explore vertex e in G (with finishing time 14) to obtain SCC {e, a, b}. This is "correct" (for this example).
Your task is to show that student J is actually wrong!

Solution:

Create a simple directed graph with 4 vertices and 4 edges:

1→2→4
↑ ↙
3

Assuming that the vertices are ordered based on increasing vertex number, the first DFS(G) will compute the following discovery/finishing time for each vertex:

1 (1/8) → 2 (2/7) → 4 (5/6)
↑ ↙
3 (3/4)

Now, if we re-run DFS(G) again based on increasing finishing time f(u), i.e. vertex 3 first (finishing time 4), then we will find {3, 1, 2, AND wrongly 4} as part of one SCC whereas there is no way we can go from 4 to {1, 2, 3}.

If we run DFS on G^T based on decreasing finishing time f(u), i.e. vertex 1 first (we get SCC {1,3,2}) and then vertex 4 (we get SCC {4}).

D3. (Lecture 8+11+12 review, Exercise 35.1-4 of CLRS, slightly edited)
Greedy algorithm for MIN-VERTEX-COVER for a <u>tree</u>

Give an efficient greedy algorithm that finds and optimal vertex cover for a <u>tree</u> in linear time. Show the optimal substructure and prove that your greedy choice is optimal.

Note that the decision version of MIN-VERTEX-COVER is proven to be NPC in Lecture 08 but this exercise shows us that polynomial solution exists for special case of MIN-VERTEX-COVER on tree.

Solution:

A tree has leaves and internal vertices. <u>Intuitively</u>, covering the leaves is a not a good idea as leaf only has exactly one neighbor however internal vertices can have more than one neighbors. It will be more beneficial to cover the internal vertices.

The simple Greedy Algorithm is as follows: Let the original tree be T. Pick any leaf L of T. Pick any vertex V adjacent to L and put V inside the vertex cover. We then remove all edges that are covered by V. If doing so causes a vertex to have degree 0 (no neighbor), we remove that vertex too. We now have a smaller tree T'. We repeat this process on T' until there is no more edge that is not covered. This can be implemented in linear time O(V). Remember that E = O(V) in a tree.

TA will show a few examples of this algorithm on random trees.

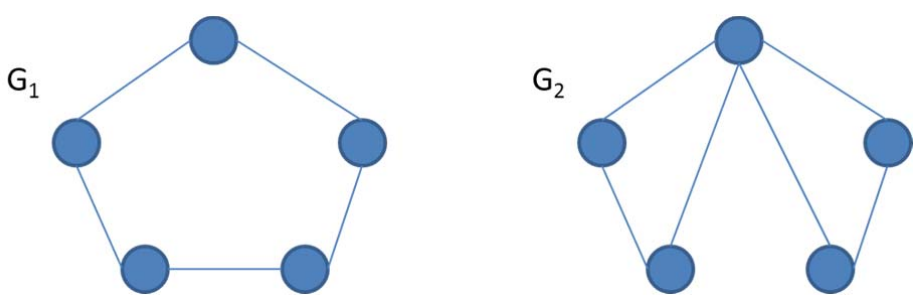
Optimal substructure: Notice that after we are done with one step of the Greedy Algorithm above, we have the subproblem (subtree) T' which is essentially the same as the original problem (tree) T but smaller.

Proof of Greedy-choice: Suppose there is an optimal vertex cover that includes at least one leaf y of T instead of the internal vertex counterpart x in T'. We have to cover this edge (x,y) with either x or y and exchanging them does not increase the vertex cover size.

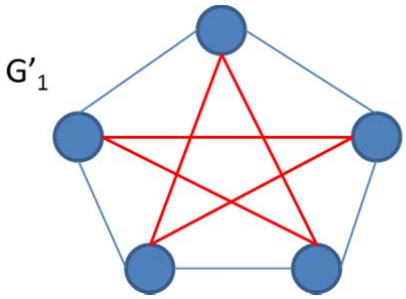
Note: MIN-VERTEX-COVER on Tree can also be solved with Dynamic Programming in O(V) too (details omitted).

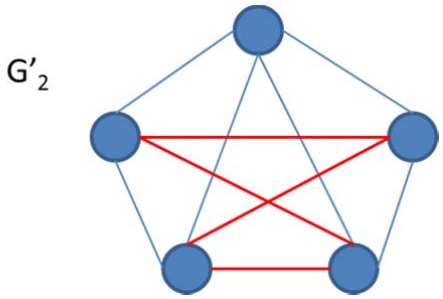
D4. (Lecture 8+12 review, Reduction from Hamiltonian-cycle to ρ-approx TSP)

a). Consider the following two instances G_1 and G_2 for Hamiltonian-cycle problem. Can you generate the transformed instances G_1' and G_2' for ρ-approx TSP using the strategies outlined in Lecture 12? What is the cost of the TSP tour in G'_1 and G'_2?



Solution:
Below, we show G'_1 and G'_2., where red edge is of weight $5\rho+1$ and blue edge is of weight 1.





The TSP for G'_1 is 5 while the TSP for G'_2 is $5\rho+5$.

b). Suppose you have a ρ-approximation polynomial time algorithm A that solves the TSP problem. Can we solve the Hamiltonian cycle in polynomial time. Please use G_1 and G_2 to illustrate your answer.

Solution:
Suppose we have a ρ-approximation polynomial time algorithm that can solve the TSP problem. Given a graph G, let G' be the transformed graph. If the minimum cost of the TSP tour in G' is at most ρn, we report G has a Hamiltonian cycle; otherwise, we report no.

For example, in G'_1, since the optimal tour is 5, the approximation algorithm will return a tour of cost at most 5ρ, hence we report G_1 has a Hamiltonian cycle.

In G'_2, since the optimal tour is of cost $5\rho+5$ which is bigger than 5ρ, we report G_2 does not have a Hamiltonian cycle.

This is an impossible scenario, thus we conclude that there is NO ρ-approximation polynomial time algorithm that can solve the general case of TSP problem.